Here is your **Experiment 11 (Kubernetes) – EASY + STEP-BY-STEP (Windows exam ready)** 👇

🧪 Experiment 11: Kubernetes Deployment (Simple)

🎯 Aim

To deploy a Docker container using Kubernetes and manage it using kubectl.

⚙️ Step 0: Requirements

👉 Install:

- Docker Desktop (enable Kubernetes ✅)
- Kubernetes
- kubectl

👉 Enable Kubernetes:

- Docker Desktop → Settings → Kubernetes → Enable

📁 Step 1: Create Project Folder

```
mkdir k8s-project
cd k8s-project
```

🐍 Step 2: Create app.py

```
notepad app.py
```

👉 Code:

```
from flask import Flask
```

```
app = Flask(__name__)

@app.route('/')
def home():
    return "Hello from Kubernetes!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

🎁 Step 3: Create requirements.txt

```
notepad requirements.txt
flask
```

📄 Step 4: Create Dockerfile

```
notepad Dockerfile
FROM python:3.9

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

CMD ["python", "app.py"]
```

🐳 Step 5: Build Docker Image

```
docker build -t k8s-app .
```

📄 Step 6: Create deployment.yml

```
notepad deployment.yml
```

👉 Paste:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment

spec:
  replicas: 1
```

```
  selector:
    matchLabels:
      app: myapp

  template:
    metadata:
      labels:
        app: myapp

    spec:
      containers:
      - name: myapp
        image: k8s-app
        ports:
        - containerPort: 5000
```

📄 Step 7: Create service.yml

```
notepad service.yml
```

👉 Paste:

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service

spec:
  type: NodePort
  selector:
    app: myapp

  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30007
```

▶ Step 8: Apply Deployment

```
kubectl apply -f deployment.yml
```

▶ Step 9: Apply Service

```
kubectl apply -f service.yml
```

🔍 Step 10: Check Pods

```
kubectl get pods
```

👉 Output: Running pod ✅

🌐 Step 11: Access Application

👉 Open browser:

```
http://localhost:30007
```

👉 Output:

```
Hello from Kubernetes!
```

🧪 Step 12: Check Services

```
kubectl get services
```

🛑 Step 13: Delete Resources (Stop)

```
kubectl delete -f deployment.yml
kubectl delete -f service.yml
```

📄 Expected Output / Result

✅ Deployment created
✅ Pod running
✅ Service exposed
✅ App accessible in browser

📌 Important Viva Points

👉 Kubernetes → manages containers
👉 Pod → smallest unit
👉 Deployment → manages pods
👉 Service → exposes app
👉 kubectl → command tool

🎯 Conclusion

Successfully deployed a Docker application using Kubernetes and accessed it via service.

⚡ VERY SHORT (Exam Writing)

1. Create app + Dockerfile
2. Build image
3. Create deployment.yml + service.yml
4. Run `kubectl apply`
5. Access app

If you want: ✅ Viva questions for Kubernetes (VERY IMPORTANT 🔥)
✅ Diagram for deployment
✅ Quick revision sheet for all experiments

Just tell 👆